

Chapter 5

Explainability

Deep neural networks excel at complex tasks but operate as opaque “black boxes”. In high-stakes fields like healthcare or autonomous driving, relying on uninterpretable models is both a practical and ethical risk. To address this, the machine learning community uses two distinct approaches: interpretability and explainability.

Interpretability refers to models that are transparent by design, such as decision trees or linear regressions, where the reasoning is natively visible. Explainability, by contrast, is post-hoc. It applies external techniques to deduce the reasoning behind a black-box model’s predictions after the fact, without altering its architecture.

This chapter covers key post-hoc explainability techniques for deep learning, including feature visualization, perturbation-based methods, and gradient-based methods like CAM and Grad-CAM. Finally, we explore rigorous evaluation frameworks—such as loyalty/inverse loyalty metrics, ROAR, and sanity checks—to ensure these explanations are mathematically sound and genuinely faithful to the model’s true logic.

5.1 Introduction

5.1.1 Why Explainability?

In recent years, artificial intelligence has advanced at a dizzying pace, with models becoming increasingly accurate, complex, and difficult to interpret. Algorithms such as deep neural networks, ensembles, and generative models offer outstanding performance, but their opacity poses a critical problem: how can we trust a system we cannot understand?

Explainability seeks to address this need. Its goal is to make AI models understandable to humans, whether end users, developers, or business leaders. This not only improves confidence in the results but also allows for the detection of errors, biases, and vulnerabilities that might otherwise go unnoticed.

The motivation behind explainability goes beyond technical curiosity. In sectors such as medicine, finance, and law, an automated decision without clear justification can have serious

ethical, legal, and social consequences. Furthermore, with regulations such as the General Data Protection Regulation (GDPR) in Europe, the ability to explain automated decisions is no longer just desirable, but mandatory.

This chapter explores why explainability is key to developing responsible and safe AI models, what techniques are currently available, and how to choose the best strategy based on the context of the problem.

5.1.2 Interpretability vs Explainability

Although the terms interpretability and explainability are often used interchangeably, in the context of artificial intelligence they represent distinct and complementary concepts.

Interpretability refers to the ability to directly understand how a model works. That is, a person can inspect the model and reason about its behavior without the need for external tools. For example, a small decision tree or a linear regression are interpretable because their rules and coefficients can be followed step by step. This type of models are called **white-box** models.

Explainability, on the other hand, focuses on generating explanations about model predictions, even if the models are not directly interpretable. For example, a deep neural network may be opaque on its own, but using some techniques, we can construct explanations that help us understand why the model made a certain decision. This type of models are called **black-box** models.

In this chapter, we will study explainability techniques for black-box models, as they are the most used in computer vision.

5.2 Visualization of Internal Features

5.2.1 Maximally Activating Patches

A common technique for interpreting convolutional networks is to analyze **what types of patterns cause certain layers or filters to activate more strongly**. This is known as searching for maximally activating patches, that is, the image fragments that provoke the greatest activation in a specific layer/channel of a CNN.

Note that when we talk about the activations of a channel we refer to a channel of a specific layer and when we talk about the activations of a layer we refer to all the channels of that layer (we can, for example, take the mean or maximum along the channels dimension to obtain a 2D activation).

The idea is simple: if a layer/channel responds strongly to a specific region of an image, that region contains information that the network considers relevant. This technique allows us to visualize and understand what the different layers/channels represent internally, helping to validate that the model is learning features consistent with the problem, or to discover if it

is focusing on irrelevant or biased details.

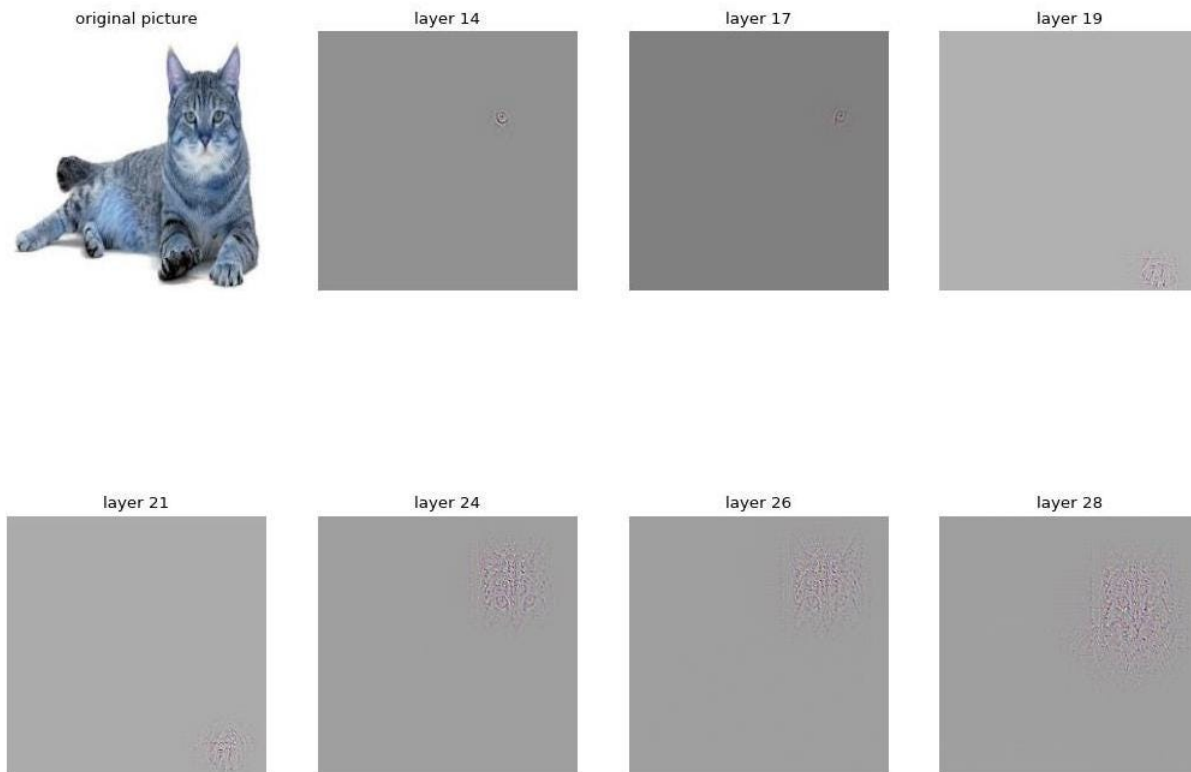


Figure 5.1: Maximally activating patches of different layers in a CNN. Layers 14 and 17 visualize the eyes of the cat. Layers 19 and 21 visualize the paws, and layers 24, 26, and 28 visualize the face.

The algorithm is the following:

1. Run an image through the network and record the activations of chosen channel/layer.
2. Upscale the activations to the shape of the original image.
3. Visualize image patches that correspond to maximal activations.

Maximally activating patches do not explain all of a CNN’s behavior, but they offer a valuable window into the network’s “inside” and are especially useful for auditing complex models and debugging their behavior.

5.2.2 Gradient Ascent

A powerful technique for understanding **what activates a specific layer or filter** in a CNN is gradient ascent. Unlike maximally activating patches, which look for real-world examples that trigger high activations, gradient ascent generates synthetic images that directly maximize the activation of a neuron, channel, or class.

The procedure is as follows: starting with an initial image (it can be noise or a base image), its pixels are iteratively adjusted using the gradient of the target activation with respect to the input. In other words, it applies an inverted version of traditional training: instead of modifying the model’s weights to minimize a loss, the image pixels are modified to maximize the desired activation.

Mathematically, this can be seen as:

$$\operatorname{argmax}_x \{ \operatorname{agg}(f(x)) - \lambda \|x\|_2 \} = \operatorname{argmax}_x \{ \mathcal{F}(x) \},$$

where x is the image we want to find, $f(x)$ the activation of the layer/channel and λ a parameter to penalize the L_2 norm of the generated image (regularization term). The term agg just means an aggregation function:

- If f refers to the final output for a class, $\operatorname{agg}(f(x)) = f(x)$, as $f(x)$ is a scalar and we cannot apply an aggregation function.
- If f refers to the activation of a channel of a layer, agg can be the mean, maximum or the norm along height and width of the activation.
- If f refers to the activation of a layer, agg can be the mean, maximum or the norm along channels, height and width of the activations of the layer.
- In some cases, it is also interesting to take the aggregation function of the absolute values. This is because large, negative terms can also be important.

To solve this problem, we can repeat for many iterations

$$x_{t+1} = x_t + \alpha \nabla \mathcal{F}(x_t),$$

where x_t is the image at iteration t , $\nabla \mathcal{F}(x_t)$ its gradient with respect to the function and α the learning rate.

The result of this process is a synthetic image that represents the “ideal stimulus” for a specific unit of the model. For example, we can generate an image that maximizes the activation of filter 23 in the third convolutional layer, or the output corresponding to the “cat” class in the final layer.

These visualizations allow us to understand what kind of abstract patterns the model has learned in different parts of the network, although the result can often be difficult to interpret directly (due to its artificial nature). To improve readability, techniques such as regularization or clipping are often applied to stabilize the process and generate more natural images.

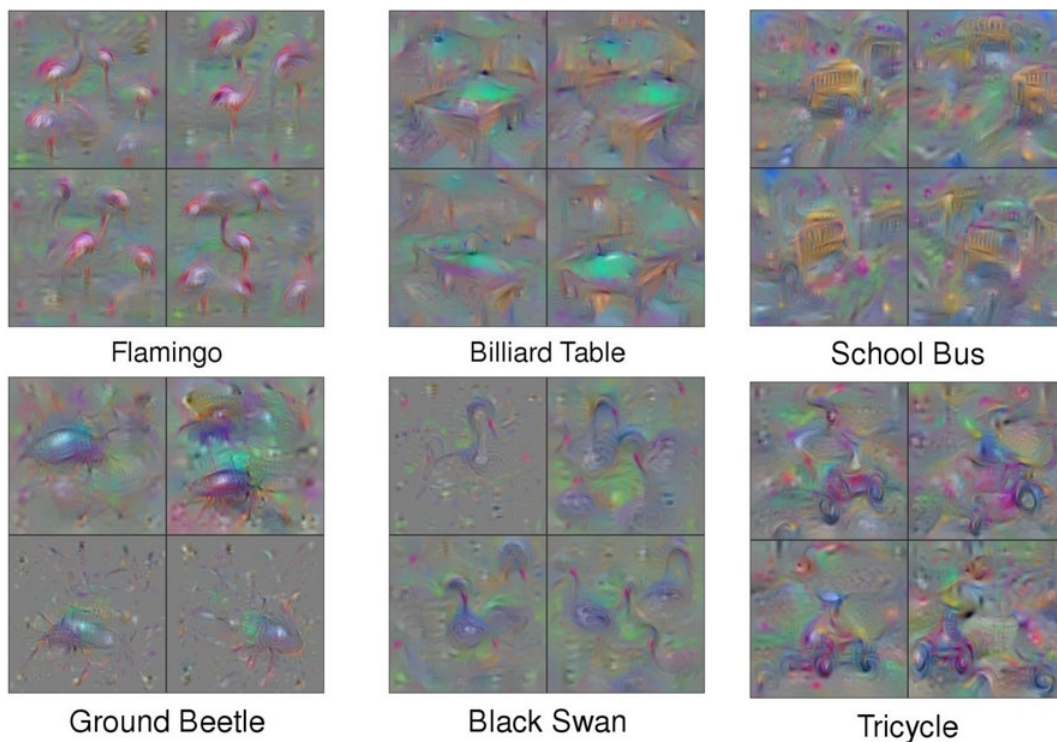


Figure 5.2: Images generated for maximizing the final score of different classes with gradient ascent.

5.2.3 Feature Inversion

Feature inversion is a technique that seeks to answer the following question: **what information is the network actually storing in its internal representations?** To do this, it attempts to reconstruct an input image from its activations in a specific intermediate layer/channel of the network.

The procedure is conceptually simple: an image is taken, its internal representation is calculated (e.g., the output of convolutional layer 4), and then an attempt is made to find a new image that, when passed through the network, produces activations as similar as possible to the original ones. In other words, an attempt is made to reverse the feature extraction process.

This is formulated as an optimization problem: starting with an initial image (noise or a base image), its pixels are adjusted to minimize the difference between the activations generated by that image and the target activations. Mathematically, we want to minimize:

$$\mathcal{F}(x) = ||f(x) - f(x_0)||_2^2 + \lambda ||x||_2,$$

where x_0 is the original image, $f(x)$ the activation of the layer/channel and we use again λ as a regularization term. A similar iterative process as in gradient ascent can be followed to solve the problem.

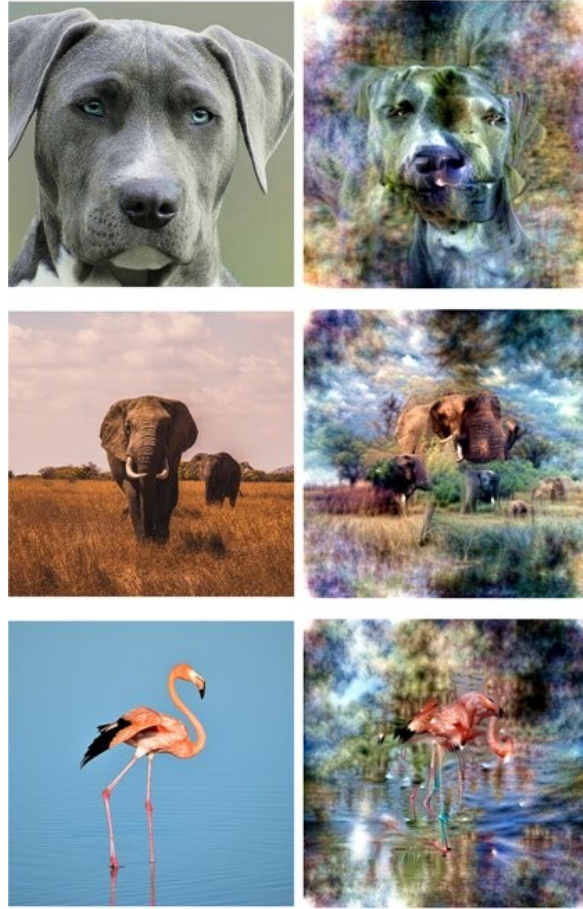


Figure 5.3: Original images (first column) and reconstructed images using feature inversion (second column).

The result of this process is a new image that reflects what the network “remembers” about the input at a given level of abstraction. At lower layers, reconstructions are typically very similar to the original image (with clear edges and textures), while at deeper layers, the reconstructed image becomes more fuzzy and semantic, losing specific details but preserving the overall structure.

This technique reveals how the network discards unnecessary information as it goes deeper, retaining only what is relevant to the task (e.g., general shape, position, key texture). Feature inversion is useful for auditing models, detecting whether they are memorizing irrelevant details, and understanding the degree of compression or generalization occurring at each layer of the CNN.

5.2.4 DeepDream

DeepDream is a technique developed by Google engineers that explores **what a neural network “sees” when asked to boost what it already recognizes**. Although it emerged as an artistic tool, it has also proven useful for visualizing what internal patterns activate certain neurons or layers.

The idea behind DeepDream is similar to gradient ascent, but with a key difference: instead of maximizing a specific class or activation from scratch, it starts with a real image and amplifies the activations already present. It is like telling the network: “Take what you already recognize in this image and make it stronger”.



Figure 5.4: DeepDream applied to one of the first layers (left) and one of the last layers (right). Note that in the first image more textures are remarked, while in the second image more high level objects are remarked.

The process to solve the problem is exactly the same as in gradient ascent, but now the initial image is the original image instead of noise or a base image.

As the iterative process is repeated, the image begins to transform and display patterns that the network “dreams” or amplifies: eyes, feathers, buildings, animals, etc., depending on what the network has learned during training. Hence the name: DeepDream, as if the network were imagining or hallucinating what it sees.

In terms of interpretability, this technique does not offer precise or quantitative explanations, but it is useful for gaining a visual intuition of the latent concepts the network has internalized. It is also an effective way to illustrate how a CNN represents knowledge in multiple hierarchical layers.

5.3 Perturbation-Based Methods

Perturbation-based methods are explainability techniques based on a simple but powerful idea: **hide parts of the input and observe how the output of the model changes**. If hiding a specific region significantly affects the prediction, then that region is important for the model’s decision.

In the case of images, this is typically implemented by covering blocks (patches) of pixels with a neutral value (such as gray or black), one by one, and evaluating how the probability of the target class changes. The result is a heat map showing which regions of the image were most relevant to the prediction.

This technique is completely independent of the model architecture and does not require access to gradients, which is why it is considered a **model-agnostic** method. This means

we can apply these techniques to any model: linear regression, random forest, neural networks... It is also intuitive and easy to visualize, although **computationally expensive** if many occlusions are applied.

Warning: Size of the patch

- The bigger the patch, the less computationally expensive the method will be, as there will be less forward passes.
- The smaller the patch, the more accurate contribution of the patch, but the method will be more computationally expensive, as there will be more forward passes. Also, if the patch is too small, it may not capture the local context of the patch.

There is not an optimal solution for this problem. We have to take into account the pros and cons of each patch size to decide our choice.

5.3.1 SHAP

General Idea

SHAP (SHapley Additive exPlanations) is a **game-theoretic** explainability method that assigns each input feature a quantitative contribution to the model's prediction. Although originally designed for tabular data, it can also be applied to images, with specific adaptations. In the case of images, each "feature" can be an individual pixel or a group of pixels (e.g., a superpixel or a patch). The goal of SHAP is to estimate how much each image region contributes to the model output for a specific class.

The algorithm is the following:

1. The image is divided into regions, usually square groups of pixels.
2. All possible versions of the image are created where some regions are present and others are hidden (as if those features did not exist). They are replaced usually with zeros.
3. For each combination of visible/invisible regions, the model output is calculated.
4. Calculation of SHAP values: Using these outputs, the individual contribution of each region is estimated, based on the weighted average of its impact over all possible combinations (Shapley values).

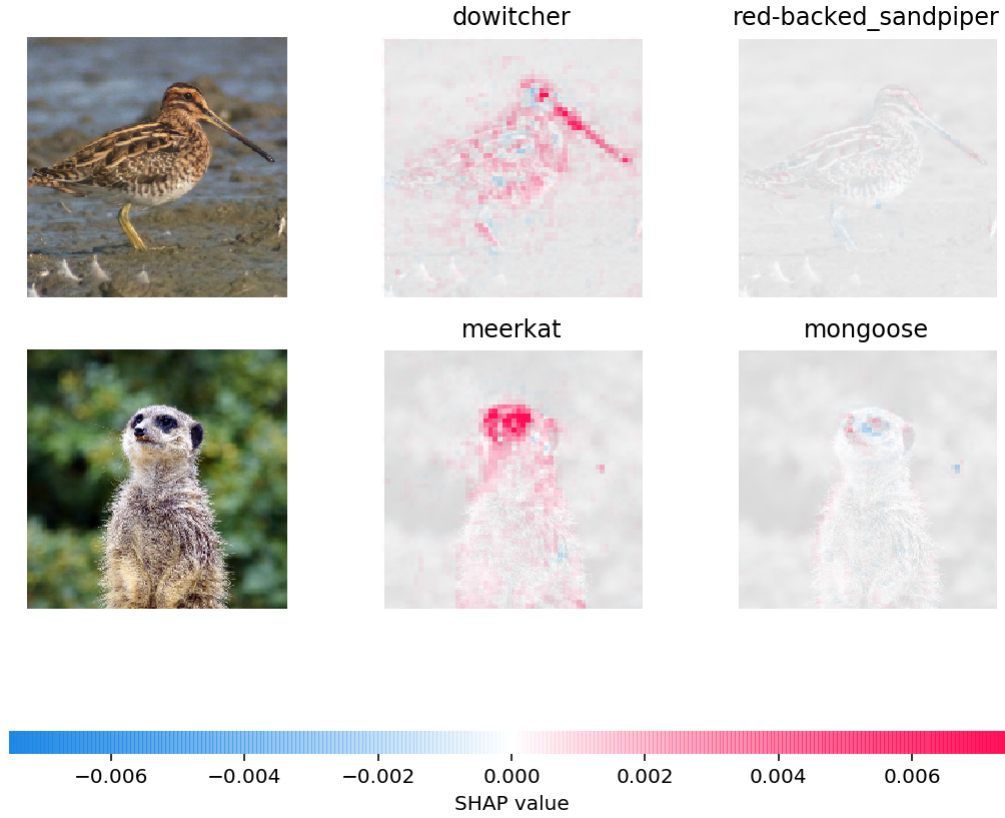


Figure 5.5: SHAP applied for different classes in two images.

It produces a heat map where each region has a SHAP value indicating whether its presence increases the probability of the target class (positive contribution, usually in red) or decreases the probability (negative contribution, usually in blue). This map allows you to see which parts of the image support or contradict the model's prediction. An example is shown in Figure 5.5.

Calculation of Shapley Values

Let's assume we divide the image into M regions and we want to calculate the contribution of each region to the prediction for a specific class of the model f . Let's also denote $N = \{1, \dots, M\}$. For each subset of regions $S \subseteq N - \{i\}, i = 1, \dots, M$, the model prediction is evaluated when only the regions of S are visible, and it is compared to the prediction when region i is also included. This part corresponds to the absolute value (right term) of the formula.

The general formula for the SHAP value for region i is:

$$\phi_i = \sum_{S \subseteq N - \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} |f(S \cup \{i\}) - f(S)|.$$

The combinatorial coefficient (the fraction, first term of the formula) is the Shapley weight, which guarantees fairness in the assignment (each order of appearance has equal probability).

The second term of the formula means the difference in the prediction with and without region i . SHAP values are based on the idea that each possible order of appearance of regions is equally likely. Therefore, to calculate the average contribution of a region i , we must consider all possible positions in which it could be added and average their marginal contributions fairly.

Example: Calculation of Shapley values

Suppose the regions are A , B and C . Let's calculate the Shapley weight for A . We have that:

- $S \subseteq N - \{A\} \equiv S_0 = \emptyset, S_1 = \{B\}, S_2 = \{C\}, S_3 = \{B, C\}$. Note that $\{C, B\}$ is not included because the order does not matter.
- The weight for S_0 is $w(S_0) = 0!(3 - 0 - 1)!/3! = 1/3$.
- Using the same formula, $w(S_1) = 1/6$, $w(S_2) = 1/6$ and $w(S_3) = 1/3$.

An intuitive way of understanding the weight is writing all possible permutations:

- ABC
- ACB
- BAC
- BCA
- CAB
- CBA

We can deduce:

- The probability that the first is A is $2/6 = 1/3$, so $w(S_0) = 1/3$.
- The probability that the second is A and the first is B is $1/6$, so $w(S_1) = 1/6$.
- The probability that the second is A and the first is C is $1/6$, so $w(S_2) = 1/6$.
- The probability that the third is A and the first and second are, B and C or C and B , is $2/6 = 1/3$, so $w(S_3) = 1/3$.

What Is Done in Practice?

In practice, evaluating every combination may be prohibitive, so **KernelSHAP** estimates the Shapley values using a weighted linear regression. Suppose that the image is divided into M superpixels and that we want to explain the scalar output $f_c(x)$, such as the logit or probability of class c .

A **coalition** S is a subset of visible superpixels. Let x_S be the image in which regions in S retain their original pixels and all other regions are replaced by a chosen baseline, such as zero, the dataset mean, or a blurred image. Its coalition value is

$$v(S) = f_c(x_S).$$

Therefore, $v(\emptyset) = f_c(x_\emptyset)$ is the prediction for the fully masked image, not for an empty tensor. Similarly, $v(\{1, \dots, M\}) = f_c(x)$ is the prediction for the original image.

Each coalition is represented by a binary multi-hot vector $z \in \{0, 1\}^M$, where $z_i = 1$ if region i is visible. For example, $z = (1, 0, 1)$ represents $S = \{1, 3\}$. KernelSHAP generates one regression sample for each selected coalition:

$$\underbrace{z}_{\text{surrogate input}} \longrightarrow \underbrace{v(S) = f_c(x_S)}_{\text{surrogate target}}.$$

The targets are predictions from the original model, not ground-truth labels, and the original model is not retrained.

These samples are used to fit the linear surrogate

$$g(z) = \phi_0 + \sum_{i=1}^M \phi_i z_i$$

with the Shapley kernel

$$\pi(S) = \frac{M-1}{\binom{M}{|S|} |S| (M-|S|)}.$$

The kernel gives greater weight to very small and very large coalitions, which are especially informative about the effect of adding or removing a region.

The cases $|S| = 0$ and $|S| = M$ make the kernel singular, so they are imposed as exact constraints rather than evaluated with the formula:

$$\phi_0 = v(\emptyset), \quad \sum_{i=1}^M \phi_i = v(\{1, \dots, M\}) - v(\emptyset).$$

Consequently,

$$f_c(x) = \phi_0 + \sum_{i=1}^M \phi_i.$$

Thus, each ϕ_i is the contribution of region i relative to the selected masking baseline. With all coalitions, the weighted least-squares solution recovers the Shapley values. Sampling coalitions gives an approximation, while regularization or feature selection may introduce bias. The Shapley kernel is therefore an essential part of KernelSHAP, not an optional weighting.

5.3.2 Occlusion

Occlusion explains a selected scalar model output $f_c(x)$, such as the logit or probability of class c , by measuring how it changes when image regions are masked. For an occluded image x' , we use the signed score

$$\Delta = f_c(x) - f_c(x').$$

A positive value indicates evidence supporting class c , while a negative value indicates opposing evidence.

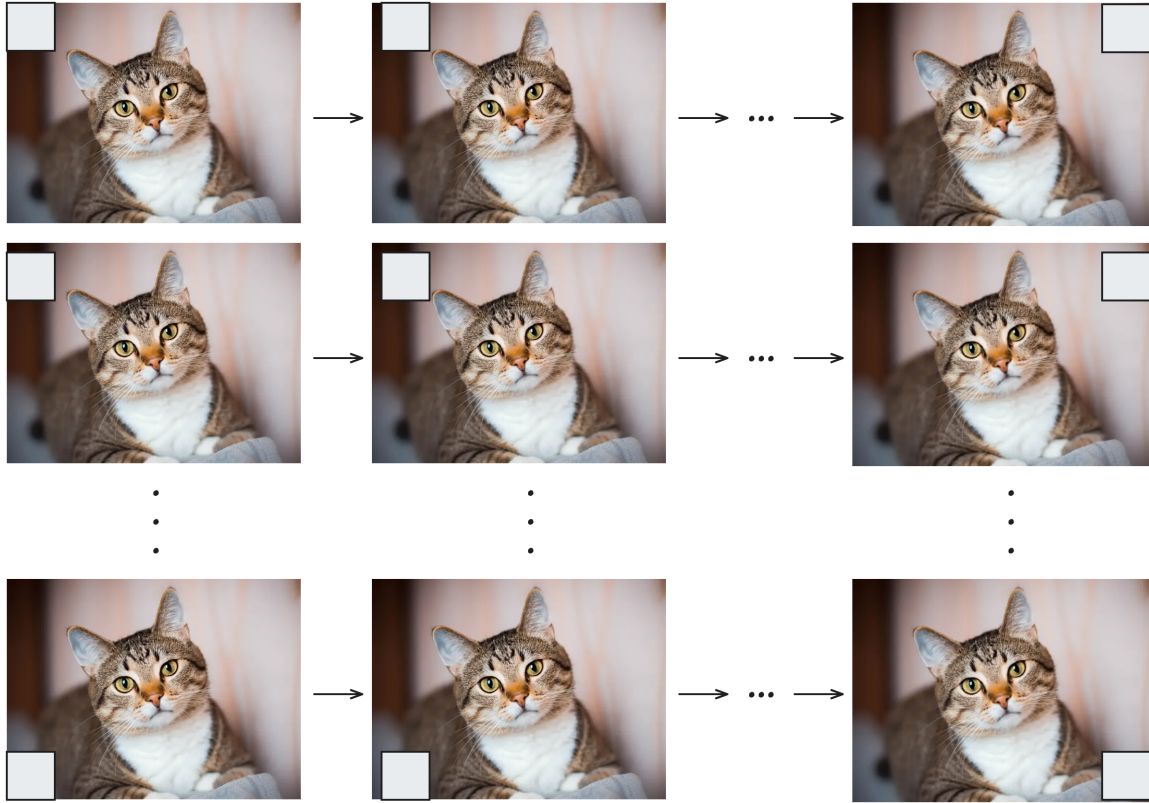


Figure 5.6: Masked pixels for each iteration in occlusion.

Using a valid $S \times S$ window with stride one, the algorithm is:

1. Choose a masking baseline and initialize an accumulated-score map A and a coverage-count map K as zero-valued $H \times W$ tensors.
2. For every position (u, v) , create x' by replacing the patch $[:, u:u+S, v:v+S]$ with the baseline.
3. Add $f_c(x) - f_c(x')$ to $A[u:u+S, v:v+S]$ and add 1 to $K[u:u+S, v:v+S]$.
4. Return the element-wise map A/K wherever $K > 0$.

The count map is necessary because overlapping windows contribute several observations to each pixel, and boundary pixels are covered fewer times. With stride one, there are $(H - S + 1)(W - S + 1)$ occluded forward passes, plus one pass for $f_c(x)$ (note it can be only computed only once as first and saved in memory). Window size, stride, and baseline can be changed; larger windows produce coarser maps.

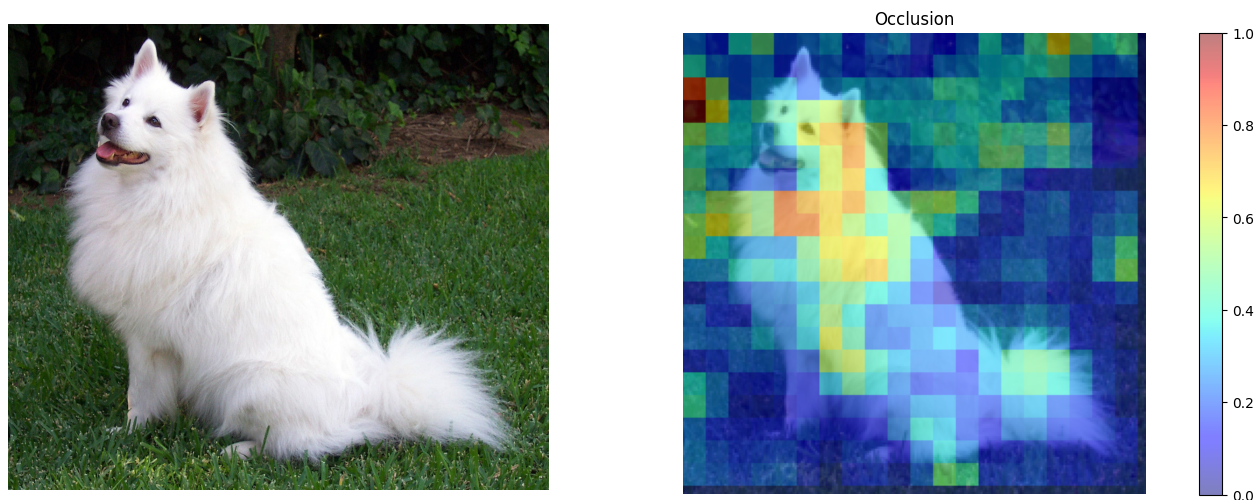


Figure 5.7: Occlusion applied to a dog. The original image is on the left and the signed target-score decrease is on the right; higher values indicate stronger supporting evidence.

5.3.3 RISE

RISE (Randomized Input Sampling for Explanation) is a black-box method that evaluates a scalar class score $f_c(x)$ on randomly masked versions of an image. Given N masks, its Monte Carlo estimator is

$$\hat{S}_c(x) = \frac{1}{Np} \sum_{n=1}^N f_c(x \odot M_n) M_n,$$

where M_n represents the n -th mask and each pixel has probability p of being masked. Thus, masks that preserve evidence for class c receive larger weights.

The algorithm is:

1. Generate a mask $M_n \in \mathbb{R}^{H \times W}$ as described below.
2. Compute the score $f_c(x \odot M_n)$ (it's a scalar) and multiply by M_n (it has the same spatial dimensions as x) to sum only on the present positions. Note that this is an element-wise multiplication.
3. Repeat N times and return the sum of this values divided by Np .

The masks are generated as follows:

1. Sample a binary $s \times s$ grid from a Bernoulli(p) distribution.

2. Let $c_H = \lceil H/s \rceil$ and $c_W = \lceil W/s \rceil$. Bilinearly upsample the grid to $(s+1)c_H \times (s+1)c_W$.
3. Take a random $H \times W$ crop using offsets in $\{0, \dots, c_H - 1\}$ and $\{0, \dots, c_W - 1\}$. This prevents all masks from sharing the same coarse-grid alignment.

The coarse grids are binary, but bilinear interpolation produces masks with values in $[0, 1]$; accumulation therefore uses the mask values rather than simply counting unmasked pixels. Values such as $N = 4000$, $s \in \{7, 14\}$, and $p = 0.5$ are example experimental settings, not universal defaults.

Note

A simpler way of generating the mask is directly sampling from a Bernoulli(p) distribution a $H \times W$ mask, but it usually produces less accurate results.

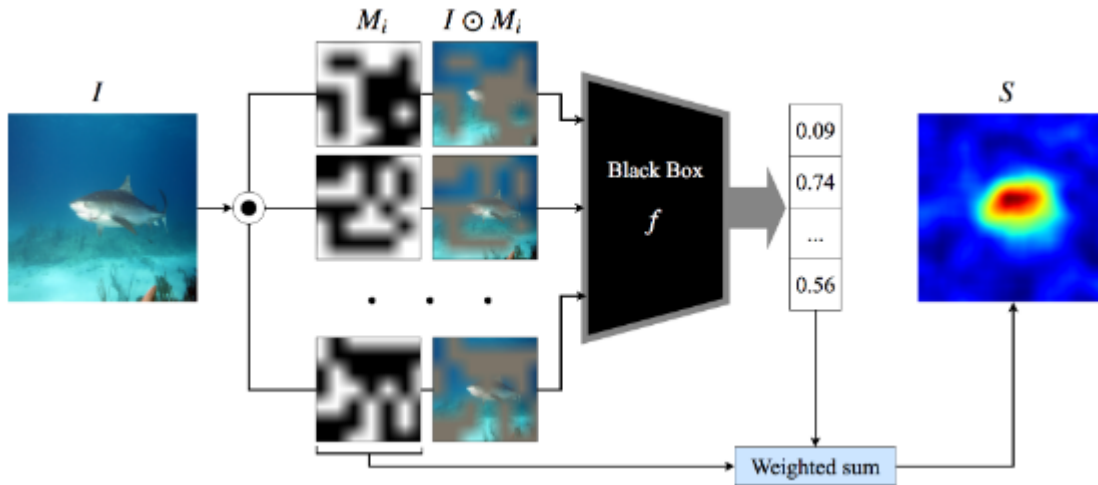


Figure 5.8: RISE applied to an image. The saliency map is a score-weighted average of the sampled masks.

5.4 Gradient-Based Methods

Gradient-based methods seek to explain a prediction by examining how the model's output varies when the input values are slightly perturbed. In other words, they measure the sensitivity of the output to each pixel in the image.

The central idea is simple: if a small change in a pixel produces a large change in the output (for example, in the probability of a class), that pixel is important for the decision.

They are **model-specific** methods, that is, they can only be applied to neural networks. However, they are much more **efficient** than occlusion methods, as usually only one forward pass and one backward pass is needed.

5.4.1 Saliency Maps

The **Saliency Map** was one of the first methods proposed to visualize which pixels in an image are most relevant to a neural network's decision. The idea is to calculate how sensitive the model's output is to each input pixel.

Given a classification model $f(x)$ and an input x (e.g., an RGB image) with shape $3 \times H \times W$, the saliency map for class c is defined as the absolute value of the maximum value of the gradient of the output for class c with respect to each pixel in the image along the channels dimension:

$$S_{i,j}(x) = \max_{k \in \{1,2,3\}} \left| \frac{\partial f_c(x)}{\partial x_{k,i,j}} \right|.$$

Note that $x_{k,i,j}$ is the pixel value at row i , column j and channel k and $S(x) \in \mathbb{R}^{H \times W}$.

This produces a map the same size as the image (with just one channel), where each value indicates how much a small change in that pixel would affect the model's output. High values represent pixels to which the model is very sensitive, so they are relevant for the prediction. Low values represent pixels that barely influence the model's decision. The resulting map can be viewed as a grayscale image (or overlaid with color) that highlights the areas of greatest influence.

Some cons are that sometimes it can be noisy and we do not have a sense of direction (positive or negative impact for the prediction). However, it is easy to implement and computationally efficient.

5.4.2 Input x Gradient

Input x Gradient is an attribution method that seeks to identify which parts of the input contribute most to the model's prediction by combining the magnitude of the input with the model's sensitivity to that input. The intuition is simple: if a pixel has a high value and the gradient (derivative of the output with respect to that pixel) is also high, then that pixel has a strong influence on the prediction.

The mathematical definition is very similar to the saliency map:

$$S_{i,j}(x) = \max_{k \in \{1,2,3\}} \left| x_{k,i,j} \cdot \frac{\partial f_c(x)}{\partial x_{k,i,j}} \right|.$$

It has the same pros and cons as the saliency map. In addition to this, sometimes it can be more informative than the saliency map, especially for inputs with many zeros or low intensity, but it always penalizes pixels with low values, and this can sometimes be incorrect.

5.4.3 Guided Backpropagation

Guided backpropagation is a gradient-based technique that aims to generate sharper and less noisy saliency maps than pure gradients. It modifies the backpropagation process to block

negative gradients in the ReLU layers during activation and backpropagation. Thus, only positive gradients propagate through neurons that were also active in the forward direction. An example is shown in Figure 5.9.

The definition of $S(x)$ is the same as in the saliency maps, but with the modified backpropagation of ReLU layers.

This method is computationally efficient and usually produces cleaner, more focused saliency maps. However, it is not faithful to the true attribution: it does not respect the causal structure of the model.

5.4.4 DeconvNet

DeconvNet is a technique originally proposed to visualize which parts of an image trigger the activation of certain neurons in a CNN. Although its name suggests “deconvolution”, it does not actually mathematically reverse convolutions, but rather performs modified backpropagation to produce interpretable visualizations. Unlike Guided Backpropagation, which requires both the forward activation and the backward gradient to be positive, DeconvNet only filters on the incoming gradient. This allows us to visualize which input patterns excite an internal neuron, regardless of whether the neuron was active in the forward pass. Backpropagation process is shown in Figure 5.9.

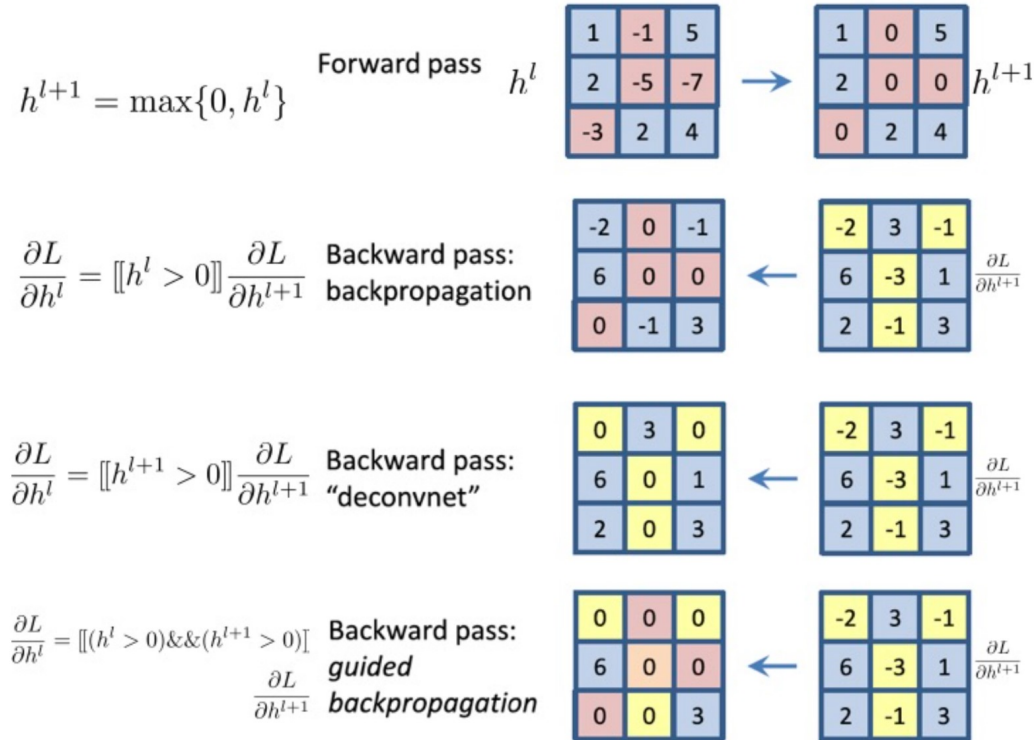


Figure 5.9: Differences in ReLU backpropagation for saliency map, DeconvNet and guided backpropagation.

Figure 5.9 summarizes the differences in the backward pass of ReLUs using different methods:

- The first part refers to the forward pass of a ReLU layer. Negative values are set to zero.
- The second image is the standard backpropagation. Gradient only propagates in positions that were not zero in the forward pass.
- The third image is the backpropagation for DeconvNet. Gradient can propagate in positions that were zero in the forward pass, but now only positive gradients propagate.
- The fourth image is the backpropagation for guided backpropagation. It is the same as standard backpropagation, but now only positive gradients propagate. It can be seen as the combination of standard backpropagation plus DeconvNet backpropagation.

5.4.5 SmoothGrad

SmoothGrad is an explainability technique that improves the quality of explanations generated by gradient methods, such as the classic saliency map. Its goal is to reduce noise and make visualizations more interpretable. It can be applied to any gradient-based method.

Direct gradient maps are often noisy and difficult to interpret due to the model's high sensitivity to small input perturbations. SmoothGrad reduces this noise by averaging gradients computed over multiple noisy versions of the original image.



Figure 5.10: SmoothGrad applied to a dog. The original image is on the left and the explanation map on the right.

The **algorithm** is the following. Given a model f , an image x and a class c :

1. N perturbed images are generated with Gaussian noise: $x_i = x + \mathcal{N}(0, \sigma^2) \quad \forall i = 1, \dots, N$.
2. For each perturbed image x_i , its explanation $S(x_i)$ is calculated with the desired method. We can choose every method we want.

3. Return $S(x) = \frac{1}{N} \sum_{i=1}^N S(x_i)$. Be careful with the mean if the explanations are not absolute values, as this way signs can cancel.

Typically N is between 10 and 100 and σ is between the 10% and 20% of the range of pixel values (e.g., if the pixels are in $[0, 1]$, then σ is between 0.1 and 0.2).

It reduces the noise of other techniques, but it is more computationally expensive, as N forward and backward passes need to be done. Even if we try to vectorize the process, there is a batch size limit because of the memory of the computer. An example is shown in Figure 5.10.

5.4.6 Integrated Gradients

Integrated Gradients (IG) is a gradient-based explainability method that seeks to attribute the importance of each pixel (or input feature) to the model's prediction more robustly than direct gradients. As SmoothGrad, it can also be applied to any gradient-based method.

IG integrates gradients along a path from a base reference to the original image. Let f be a model, x an input, and x' the reference. The explanation is defined as:

$$\text{IG}(x) = (x - x') \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x} d\alpha.$$



Figure 5.11: Interpolation of an image for different values of α . The idea is to compute the saliency map for each interpolated image and then return the averaged results.

However, in practice we can not integrate, but remember from calculus that integrals are just sums! Therefore, we can redefine:

$$\text{IG}(x) = (x - x') \frac{1}{M} \sum_{k=1}^M \frac{\partial f\left(x' + \frac{k}{M}(x - x')\right)}{\partial x},$$

where M is the number of steps in the interpolation, typically between 50 and 300. The part of multiplying by $(x - x')$ is for the attribution to satisfy the completeness axiom, i.e.,

that the sum of all values will be equal to the difference between the original image and the baseline.

This is a very strong and robust method, but it is more computationally expensive. Moreover, the bigger M , the better it approximates the integral, but again the more computationally expensive it is.

The general idea is to average the gradients obtained from the baseline image to the original image and multiply this by the difference of both images. Figure 5.11 shows an example of an interpolation.

Note

In the original paper, they suggest that the baseline image should be one that produces an output of 0 for the class we are interested in. They call it “absence of signal”. Some typical examples (although they may not always produce an output of 0) are a completely black image, the mean of the pixels along the channels or the original image blurred.

5.5 Ad Hoc Methods

Ad hoc methods are techniques specifically designed to exploit the **internal structure** of certain models, especially deep convolutional networks (CNNs). Unlike general gradient or perturbation-based methods, these approaches rely on specific layers and operations of the network to generate interpretable visualizations, typically in the form of heat maps over the input image.

They are **model-specific** methods, and more restrictive than gradient-based methods, as they can only be applied to a subset of CNNs that have some particular structure. They are also much more **efficient** than occlusion methods, as usually only one forward pass and one backward pass is needed.

5.5.1 CAM

CAM (Class Activation Mapping) is a visualization technique that allows you to identify **which regions of an image have been most relevant for a convolutional network to make a class prediction**. CAM requires an architecture with:

- A final convolutional layer followed directly by
- A GAP (Global Average Pooling) layer and a linear classification layer.

Let’s suppose a CNN that, before the classification layer, produces a set of n activation maps $A_k(x)$ with shape $h \times w$, where $k = 1, \dots, n$ indexes the channels. Then the network applies:

1. Global Average Pooling:

$$F_k(x) = \frac{1}{hw} \sum_{i=1}^h \sum_{j=1}^w A_{i,j,k}(x) \quad \forall k = 1, \dots, n.$$

2. Classification layer (output for class c):

$$f_c(x) = b_c + \sum_{k=1}^n w_k^c F_k(x),$$

where w_k^c is the learned weight that connects channel k with class c .

Doing some math, we have that

$$f_c(x) = b_c + \sum_{k=1}^n w_k^c \left(\frac{1}{hw} \sum_{i=1}^h \sum_{j=1}^w A_{i,j,k}(x) \right) = b_c + \frac{1}{hw} \sum_{i=1}^h \sum_{j=1}^w \sum_{k=1}^n w_k^c A_{i,j,k}(x).$$

This suggests that a **class activation map** $M_c(i, j) \in \mathbb{R}^{h \times w}$ can be constructed as:

$$M_c(i, j) = \sum_{k=1}^n w_k^c A_{i,j,k}(x).$$

This map indicates the importance of each spatial position (i, j) for class c . By **interpolating** it back to the original image size, we obtain an interpretable visualization of “where the network looks” to classify an image as class c .

This method is intuitive and efficient, but the most obvious limitation is that it can only be applied to a few networks. Finally, a graphical representation is shown in Figure 5.12.

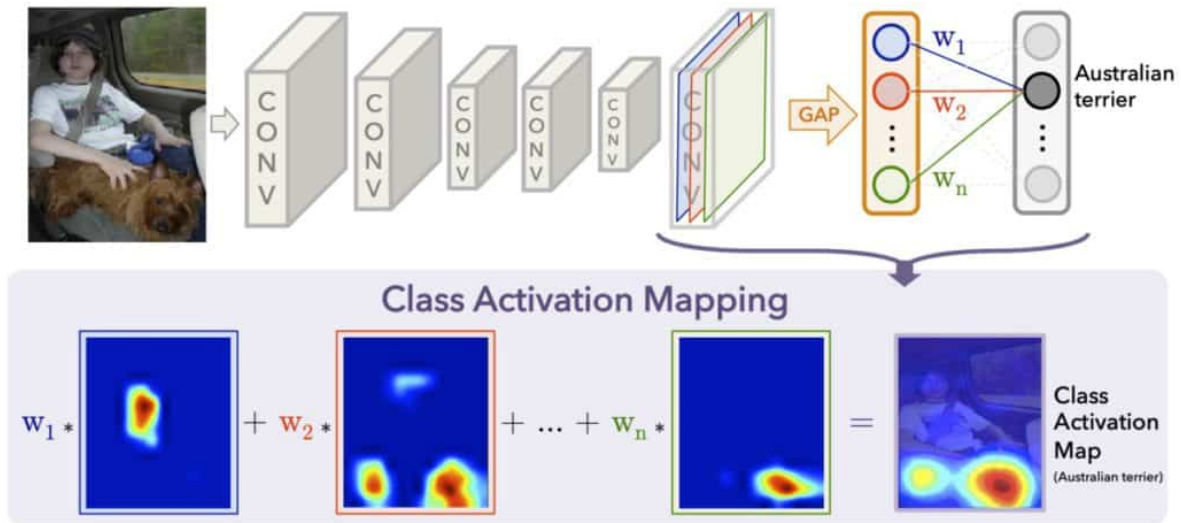


Figure 5.12: Graphical representation of CAM.

5.5.2 Grad-CAM

Grad-CAM is a **generalization of CAM** that allows activation maps to be generated in any modern CNN, without requiring a special architecture. Its goal remains the same: to visualize which image regions have most influenced the prediction of a given class.

Given a model $f(x)$, an input image x , and a class of interest c , proceed as follows:

- Select a convolutional layer (usually the last one) and obtain its activation maps $A_k(x)$, where k is the channel (feature map). We will assume there are n channels and the shape of the activation is $h \times w$.
- Calculate the gradients of the model output for class c with respect to these activation maps:

$$\frac{\partial f_c(x)}{\partial A_k(x)} \quad \forall k = 1, \dots, n.$$

- Average those gradients over the (i, j) spatial space to obtain a weight per channel:

$$\alpha_k^c = \frac{1}{hw} \sum_{i=1}^h \sum_{j=1}^w \frac{\partial f_c(x)}{\partial A_{k,i,j}(x)}.$$

- Combine the activated maps using these weights (ReLU ensures that only regions positively associated with the c class are displayed):

$$M_c(i, j) = \text{ReLU} \left(\sum_{k=1}^n \alpha_k^c A_{i,j,k}(x) \right).$$

M_c is a heat map of the same resolution as the activation maps. By interpolating it to the original image size, it can be overlaid to visualize relevant regions.

This method is also intuitive (it is like a variant of CAM) and computationally efficient, and it can be applied to any convolutional network. However, it can not be applied to traditional neural networks, for example when we work with tabular data, not images.

5.5.3 Guided Grad-CAM

Guided Grad-CAM is a technique that merges two popular explainability methods in convolutional networks:

- Grad-CAM, which generates a localized, but blurry (low-detail) activation map.
- Guided backpropagation, which produces a fine-grained, high-resolution map with little contextual information.

The idea is to multiply both maps to obtain a visualization that is spatially accurate, detailed at the pixel level and relevant to the predicted class.

Guided Grad-CAM is simply the element-by-element multiplication of both the maps obtained by Grad-CAM and guided backpropagation. Grad-CAM tells you which regions are important, guided backprop tells you which pixels are important and, therefore, guided Grad-CAM tells you which important pixels are in the relevant regions.

5.5.4 Attention as Explainability

Some models, like Vision Transformers (ViTs), use **attention** on the inputs to obtain the final prediction. Although it may be tempting to use attention weights as importance maps, there are several reasons why this is problematic:

- They are not necessarily causal: high attention weights do not imply that one part of the input causes the output. They can be spurious correlations or training artifacts.
- They are context-dependent: in models with multiple heads and layers, it is difficult to attribute meaning to a single attention map without considering more complex interactions.

Attention can be useful in some simple models, such as visual attention in classification or captioning tasks. In more complex models, if accompanied by sensitivity analyses, counterfactuals, or intervention techniques, attention maps can be a clue.

In conclusion, attention is not explanation and **“saying that attention is explainability is like saying that correlation is causation”**. It’s an accurate analogy: both can be clues, but they do not guarantee true interpretability. Using attention as an explanation requires caution, additional validation, and a clear understanding of its role in architecture.

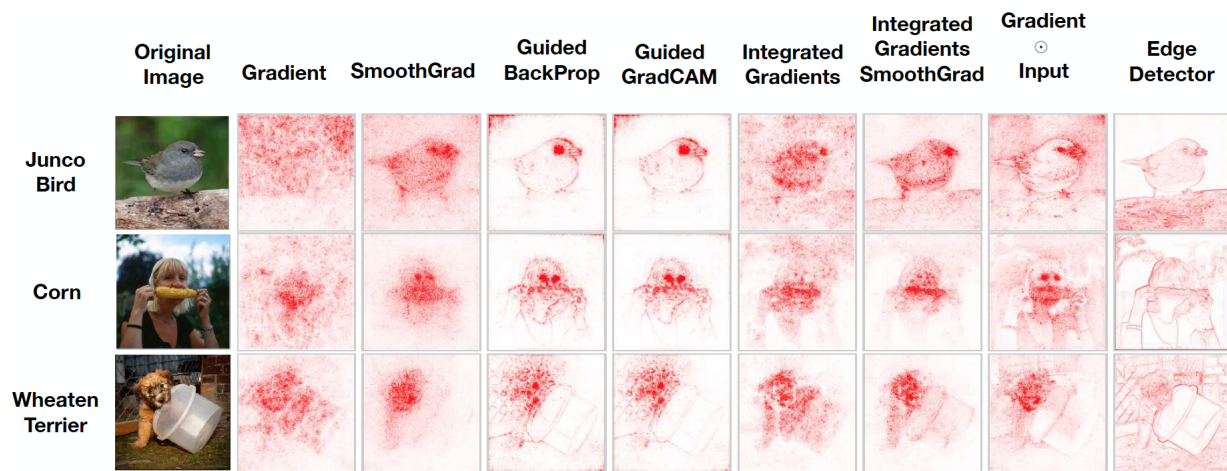


Figure 5.13: Different explainability techniques.

To end this section Figure 5.13 shows some techniques for three inputs. Note that the maps are shown themselves alone instead of displayed above the original image.

5.6 Evaluation

It is not enough to generate eye-catching visualizations: an explainability technique must be reliable, consistent and useful. To achieve this, it is essential to evaluate its quality using objective metrics or specific tests. This evaluation can be approached from several angles. This section presents the main tools and criteria used to assess the quality of an explanation, both quantitatively and qualitatively.

5.6.1 Common Problems

- **Unstable gradients:** Gradients can be extremely sensitive to small changes in the input. This means that small movements in a pixel value can lead to large changes in the magnitude or even flip the sign of a gradient. This can lead to saliency maps that are hard to interpret, as they appear speckled or random in regions that should probably be smooth or semantically meaningful. SmoothGrad has been shown to help. It usually reveals more consistent patterns.
- **Rapid sign swings:** A related problem is that we can often see rapid swings in the signs of the gradients. This has to do both with the sensitivity of the gradients and the nonlinearity of neural networks. Ultimately, this emphasizes the point that we should be more careful when the absolute value of the gradients is not used.
- **Irrelevant information:** The previous two problems could be summarized as one, and that is noisy gradients. Another factor that contributes to noisy saliency maps is that often gradients can contain information that is not specific to the target logit. This can come from pixels in the background or those from features associated with other classes. Guided backpropagation can clean up saliency maps using the ReLU masking, as it suppresses negative gradients during backpropagation, leading to a visually sharper saliency map that highlights the features used by a model. However, this method is not class discriminative, meaning it may still highlight features useful for multiple classes, not just the target one. Grad-CAM is a more reliable class discriminative method. We can combine both methods with Guided Grad-CAM.
- **Saturated gradients:** In some cases, the gradients of important pixels can still be small or even zero. During backpropagation, if the input to the ReLU function has a negative value, then the gradients will be zero, regardless of how large the negative input is. Integrated Gradients can solve this problem, because we observe how gradients change over a path.
- **Vanishing gradients:** In large networks, the gradients of earlier layers can become small because they are obtained by iteratively multiplying gradients of deeper layer, which are small values themselves. Methods like Grad-CAM, that only use gradients of deeper layers, can solve this problem.
- **Lack of context:** Gradients reflect local sensitivity. That is, how a small change in an input pixel would impact the output. This does not capture how specific regions of

the image contribute to a prediction. Integrated Gradients and SmoothGrad can solve this, because it changes all pixels at the same time. Grad-CAM also mitigates this problem, as it usually relies on deeper layers, which contain simpler features and more informative pixels.

5.6.2 Loyalty and Inverse Loyalty

The loyalty and inverse loyalty metrics are two popular ways to evaluate the fidelity of an explainability technique. That is, how well it truly reflects the importance of the identified regions relative to the model's prediction.

Loyalty measures how well the regions identified as important by an explanation match what actually influences the model's prediction.

The intuition is that if you remove what the explanation says is important, the prediction should change significantly. A good explanation will cause a rapid drop in the predicted probability for the target class. This is usually represented as a curve: percentage of removed pixels vs. difference with the model output. To quantify it, the area under the curve (AUC) is typically used. The closer to 1, the better, because it means that the most important pixels identified were actually important for the prediction.

The procedure is the following:

1. Start with the original image x and its prediction $f_c(x)$ for class c .
2. The pixels (or regions) are ordered from highest to lowest importance according to the explanation map.
3. These regions are progressively removed (e.g., by setting them to zero).
4. For each generated image, we record how the model's output changes for class c . For example, we can define it as

$$1 - \left| \frac{f_c(x) - f_c(x')}{f_c(x)} \right|,$$

where $f_c(x)$ is the original prediction and $f_c(x')$ the prediction of the modified image.

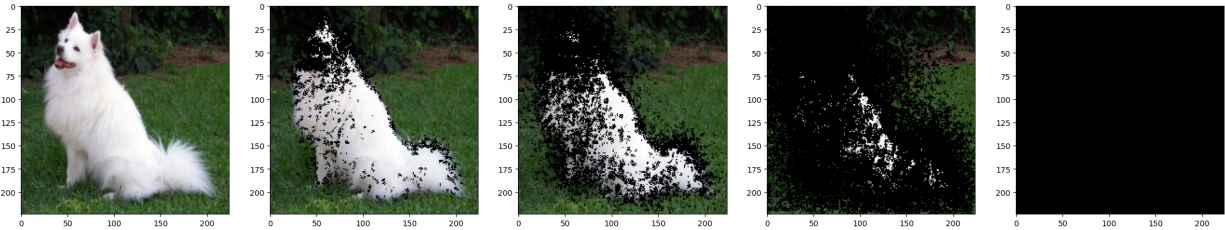


Figure 5.14: Removing most important pixels of an image.

Inverse loyalty measures how well the regions identified as irrelevant by an explanation match what actually does not influence the model's prediction.

The intuition is that if you remove what the explanation says is less important, the prediction should not change. A good explanation will cause a slow drop in the predicted probability for the target class. This is usually represented as a curve. The closer the AUC to 0, the better, because it means that the less important pixels identified were actually irrelevant for the prediction. At the end, we should see a rapid drop in the probability, as important pixels will be removed.

The procedure is the following:

1. Start with the original image x and its prediction $f_c(x)$ for class c .
2. The pixels (or regions) are ordered from lowest to highest importance according to the explanation map.
3. These regions are progressively removed (e.g., by setting them to zero).
4. For each generated image, we record how the model's output changes for class c . We can define the difference between predictions.

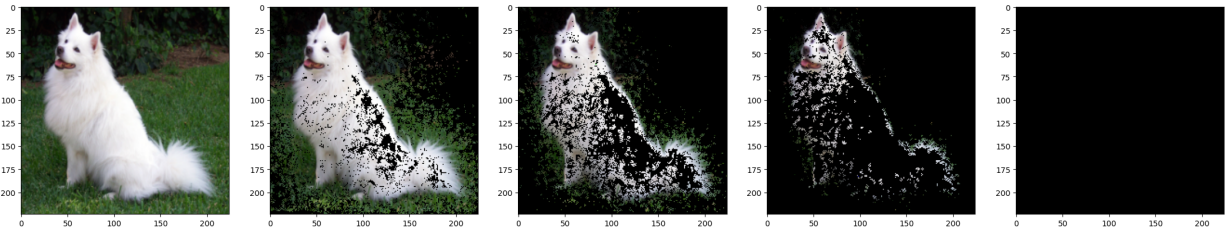


Figure 5.15: Removing less important pixels of an image.

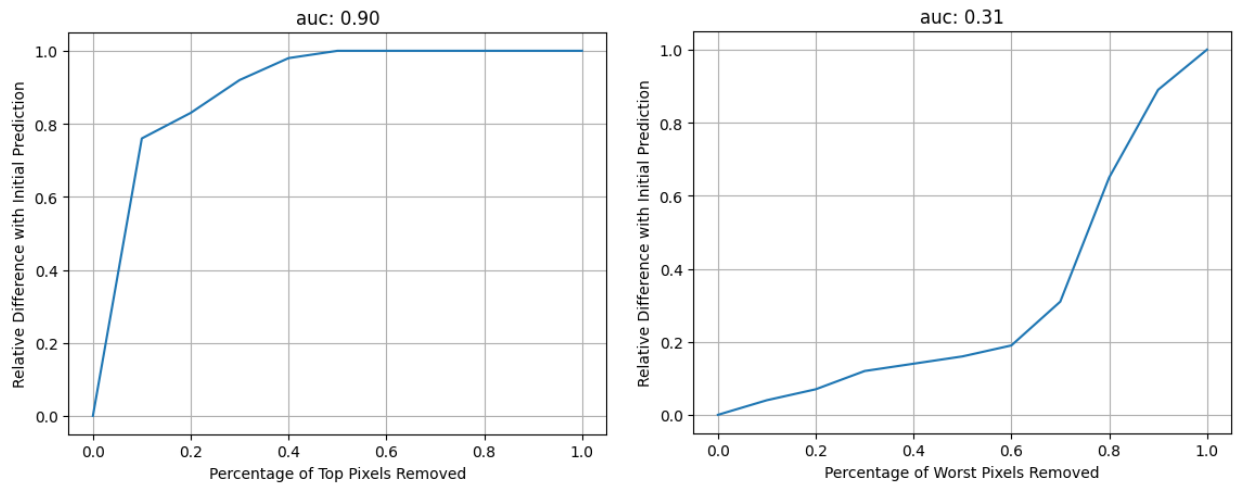


Figure 5.16: Loyalty (left) and inverse loyalty (right) curves.

Warning: Loyalty and Inverse Loyalty

Loyalty and inverse loyalty assume feature independence and cause distribution shifts of the input, i.e., inputs that the model has not seen previously in the training. Although these methods are useful, we have to be careful with the conclusions extracted. Finally, note that this problems also apply to perturbation methods, like Occlusion.

5.6.3 ROAR

The **ROAR (RemOve And Retrain)** metric is a technique for evaluating the explainability of machine learning models, especially deep neural networks. Its objective is to measure how useful the generated explanations (e.g., heatmaps of feature or pixel importance) are in understanding the model's decisions. ROAR does not directly measure the explanation, but rather its usefulness: if removing what is “important” according to the explanation causes the model to suffer, then the explanation was useful in detecting what was relevant. The biggest disadvantage is that it is computationally expensive.

These steps are followed:

1. You train an original model with a dataset.
2. You generate the explainability maps with a technique.
3. For each image, you remove the most important pixels according to each explanation. For example, you nullify the most important pixels (setting them to black, zero, or some neutral value).
4. You train a new model from scratch with this “corrupted” data for both train and test.
5. If the explanation technique is good, then the new model should perform worse because it lost crucial information. A drop in accuracy bigger than just corrupting the data randomly is interpreted as a sign of good explainability.

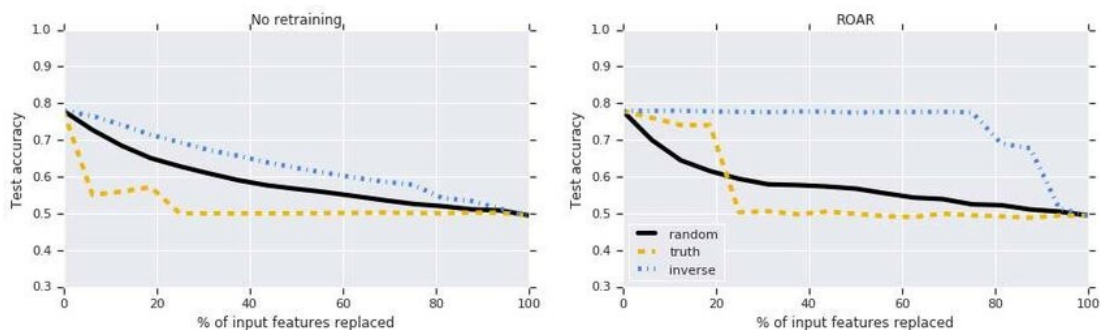


Figure 5.17: Using ROAR, we can see that after re-training most of the information is actually still present.

Figure 5.17 shows an example of what is not a good explainability technique. Orange represents the pixels removed in the order according to the explainability technique, black is a random order to use as baseline and blue the inverse of orange (from less important to most important pixels):

- If we only look at the left curve, we may think the technique is good. As we remove pixels accuracy decreases. Moreover, removing the most important pixels (orange) causes the greatest degradation in the accuracy, the less important pixels (blue) the least degradation, and removing pixels randomly (black) is an intermediate point.
- However, looking at the right curve, if we remove the same pixels, but retrain the model in each case, the accuracy is greater at first removing the most important pixels (according to the explainability technique) than random pixels! That suggests that the ranking may be less informative than it first appeared.

5.6.4 Sanity Checks

Sanity checks test whether a saliency method depends on the learned model parameters and on the relationship between the training examples and their labels. A visually appealing map may still be largely independent of what the model learned.

There are two main checks:

- **Model parameter randomization test:** Compare the original explanation with explanations produced after reinitializing model parameters. In *cascading randomization*, layers are progressively reinitialized from the output toward the input. In *independent randomization*, one layer is reinitialized at a time while all other layers retain their trained parameters. If a method is insensitive to these changes, it is unsuitable for tasks that require explanations to depend on the learned parameters, although this does not make every possible use of its visualization useless.
- **Data randomization test:** Randomly permute the training labels and train a model with the same architecture to fit this randomized labeling. The model should actually fit or memorize the random labels; otherwise, comparing its explanations with those of the original model is closer to a comparison with an untrained model. Similar explanations suggest that the method is insensitive to the relationship learned between inputs and labels.

Figure 5.18 shows *independent layer randomization*. Explanations that remain similar after individual layers are reinitialized show limited sensitivity to the parameters of those layers.

Figure 5.19 shows one data-randomization experiment on a CNN trained on MNIST. Low similarity after randomization is desirable in this test because it indicates sensitivity to the learned input-label relationship. However, this result does not establish that one method is generally better than another: it depends on the model, dataset, visualization transform, and similarity statistic used. Moreover, maps may retain similar visual structure while their

signed rank correlation changes substantially, so visual inspection and numerical metrics should be interpreted together.

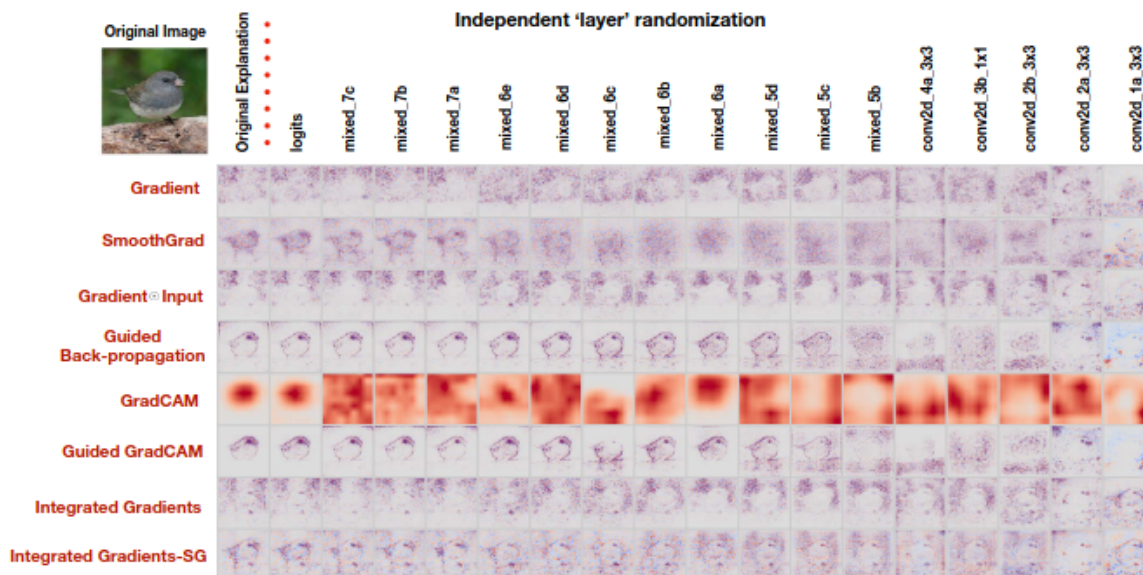


Figure 5.18: Independent layer-randomization test. Each column reinitializes one layer while retaining the remaining trained parameters.

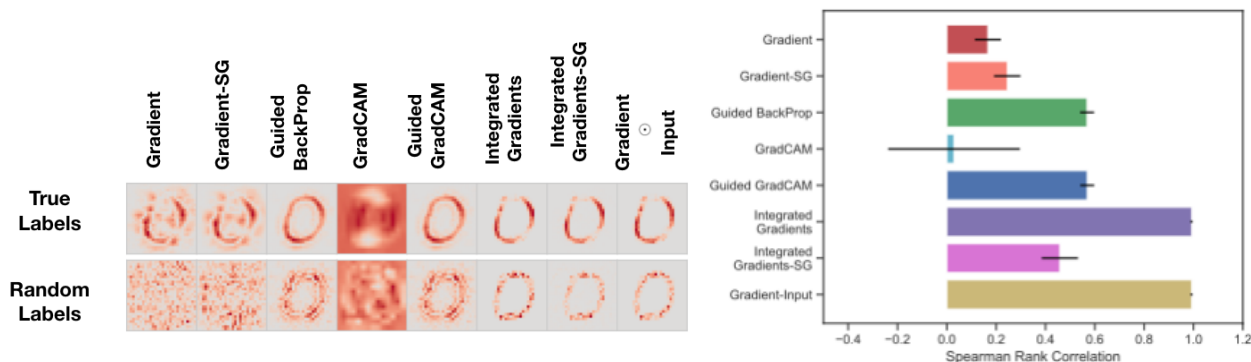


Figure 5.19: Data-randomization test on MNIST, comparing explanations from models trained with true and randomly permuted labels. On the left, there is an example of a particular explanation. On the right, there is the correlation of all the explanations.

5.7 Exercises

Exercise 1: Importance of Explainability

Give some arguments about why explainability is important. Think about particular situations where it can be useful, such as healthcare, criminal justice, or autonomous navigation.

Exercise 2: Grad-CAM

Grad-CAM was introduced as a generalization of Class Activation Mapping (CAM). Let $F_k = \frac{1}{Z} \sum_{i,j} A_{i,j,k}$ be the Global Average Pooling output, and $f_c = b_c + \sum_k w_k^c F_k$ be the class score. Prove analytically that, under the specific CAM architecture, the Grad-CAM weights $\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial f_c}{\partial A_{i,j,k}}$ are strictly equivalent to the classification weights w_k^c up to a proportionality constant.

Exercise 3: Computing Gradients

Consider a simple 1D neural network defined as a piecewise linear function: $f(x) = \text{ReLU}(x) - \text{ReLU}(x - 10)$. If our input is $x = 15$, calculate the standard gradient (saliency map) $\frac{\partial f}{\partial x}$ and explain why it fails to attribute importance to x . Then, calculate the Integrated Gradients attribution using a baseline of $x' = 0$ and show how it successfully circumvents this gradient saturation problem.

Exercise 4: Complexity of SHAP and Occlusion

Let N be the number of pixels (or superpixels) in an image. Derive the big-O time complexity for both exact Shapley values computation and a standard Occlusion approach. Explain why practical implementations of SHAP must rely on approximations like Monte Carlo sampling.

Prerequisite for Implementation Exercises:

For the following programming tasks, set up a script to load a pretrained convolutional neural network (e.g., ResNet50 in PyTorch) and a sample image dataset containing distinct classes (such as cats and dogs).

Exercise 5: CNN Feature Maps

Extract and plot the intermediate activations from various layers of the CNN. Observe and describe how the feature representations evolve as they propagate from shallow to deeper layers.

Exercise 6: Maximally Activating Patches

Identify and visualize the maximally activating patches across different layers. Alternatively, isolate the top 5 most active filters in a specific deep layer and display their corresponding input patches.

Exercise 7: Gradient Ascent

Implement a gradient ascent algorithm to synthesize an image starting from a random noise tensor. Formulate the optimization to maximize the predicted probability of a specific target class (e.g., “dog”).

Exercise 8: Feature Inversion

Perform feature inversion to reconstruct an image based solely on the representations captured at a specific layer. Use a standard input image (e.g., a “cat”) and compare the quality of the reconstruction at shallow versus deep layers.

Exercise 9: DeepDream

Apply the DeepDream algorithm to an input image by maximizing the mean activations of specific layers. Analyze the visual differences in the hallucinatory patterns generated when targeting shallow layers compared to deep layers.

Exercise 10: Shapley Values

Implement a Monte Carlo approximation of Shapley values (as commonly applied in practice). Generate a feature importance map highlighting the most critical pixels for a chosen input image.

Exercise 11: Occlusion

Develop an occlusion-based explainer that systematically slides a masking patch over the input image. Generate the resulting saliency map, noting the inverse relationship between patch size, spatial resolution, and computational execution time.

Exercise 12: RISE

Implement the Randomized Input Sampling for Explanation (RISE) algorithm. Generate a saliency map by evaluating the model on randomly masked versions of the input, weighting each mask by the resulting class probability.

Exercise 13: Saliency Map

Compute a standard Saliency Map using a single backward pass to identify the pixels that most heavily influence the model’s final prediction.

Exercise 14: Input $\times$ Gradient

Implement the Input $\times$ Gradient attribution method. Extract the gradients of the target class with respect to the input, multiply them element-wise with the original image, and visualize the resulting importance map.

Exercise 15: Guided Backpropagation

Create a Guided Backpropagation explainer. As you need to modify the backward pass, you will need to use PyTorch hooks. If you don't remember them well, I encourage you to see these two videos:

- [PyTorch Hooks Explained - In-depth Tutorial](#)
- [Visualizing activations with forward hooks \(PyTorch\)](#)

Exercise 16: DeconvNet

Reconstruct the input stimuli responsible for a specific activation using the DeconvNet approach. *Hint: You will need to carefully manage gradient hooks during the backward pass to achieve this.*

Exercise 17: SmoothGrad

Implement the SmoothGrad technique to reduce the visual noise inherent in standard gradient methods. Apply this sampling wrapper to a base method (such as standard Saliency Maps) and visualize the smoothed explanation.

Exercise 18: Integrated Gradients

Implement the Integrated Gradients algorithm. Generate an attribution map by aggregating gradients computed along a linear interpolation path from a baseline vector (e.g., a completely black image) to the actual target image.

Exercise 19: CAM

Extract the spatial importance of a prediction by implementing Class Activation Mapping (CAM).

Exercise 20: Grad-CAM

Generalize the previous approach by implementing Grad-CAM. Use PyTorch hooks to capture both the forward feature maps and the backward gradients of the final convolutional layer, without strictly requiring a GAP architecture.

Exercise 21: Guided Grad-CAM

Combine your Guided Backpropagation and Grad-CAM implementations to produce a Guided Grad-CAM visualization. Execute an element-wise multiplication of both maps to achieve a high-resolution, class-discriminative explanation.

Exercise 22: Loyalty and Inverse Loyalty

Evaluate the faithfulness of your generated explanations by computing the loyalty and inverse loyalty metrics. Plot the respective Area Under the Curve (AUC) graphs and iteratively visualize the image degradation as top pixels are masked. Discuss the out-of-distribution (OOD) shift problem associated with this evaluation.

Exercise 23: ROAR

Implement the RemOve And Retrain (ROAR) framework on a lightweight dataset (such as CIFAR-10 or MNIST). Corrupt the training dataset based on an explainer's heatmaps, retrain a new model from scratch, and compare the test accuracy drop against a baseline where input pixels are masked randomly.

Exercise 24: Sanity Checks

Conduct Sanity Checks on your explanations by implementing the Model Parameter Randomization Test and the Data Randomization Test. Try to replicate Figures 5.18 and 5.19, and compute a metric like the Structural Similarity Index (SSIM) to quantify the divergence between the original explanation and the randomized outputs.